

Technical Design Document

Run' N Swipe

M2i Gaming & Smart Tech - Promotion 2021



Run' N Swipe

TABLE DES MATIÈRES

I. Présentation du jeu :	4
1. Introduction	4
2. Fonctionnalités	5
3. Univers	6
II. Plateformes et spécifications hardware	6
III. Outils techniques utilisés	8
1. Environnement de développement	8
2. Technologies	8
IV. Pipeline de production	8
1. Production d'un modèle 3D	8
2. Production des animations	9
3. Production d'un effet sonore	9
4. Production du post-processing	10
V. Outils spécifiques	10
VI. Bibliothèques Externes	10
1. Partie Réseau	10
2. Partie Transport	11
3. Partie Génération Maps	12
4. Partie UI	12
VI. Architecture Logicielle	12
1. Côté Serveur	13
2. Côté Client	15
VII. Techniques de rendu	16
VIII. Budgets	17
IX. Performances	17
X. Mécanismes de gameplay	17
1. Génération de la map LD (Level Design)	17
2. L'Algorithme	19
XI. HUD	21
XII. Menus	22
XIII. Audio	23

XIV. Convention d'écriture de code	23
XV. Équipe et organisation	24
1. Équipe	24
2. Organisation	24
XVI. Risques techniques	24
XVII. Fonctionnement GIT	26
Annexes	27



Historique des versions du document :

Numéro de version	Date	Auteur(s)	Description
1.0	12/03/2020	Maxime MORAINÉ	Initialisation du TDD
2.0	07/04/2020	Romain SUSSEST	Mise à jour du TDD, pour la finalisation de la phase 3
2.1	03/06/2020	Romain SUSSEST	Correctifs apportés suite aux retours de M. BILEMDJIAN
2.2	22/06/2020	Romain SUSSEST Elie KARABOGHOSSIAN	Correctifs apportés suite aux retours de M. GAUDY
2.3	28/06/2020	Maxime MORAINÉ Romain SUSSEST	Finalisation du TDD pour la phase 4
2.4	17/10/2020	Elie KARABOGHOSSIAN	Modification de la partie algorithmique
2.5	20/12/2020	DERAEDT Lucas MODESTE Sullyvan	Ajout d'explication technique Correction du document
3.0	23/05/2021	DERAEDT Lucas	Mise à jour des algorithmes, système de vie et collisions

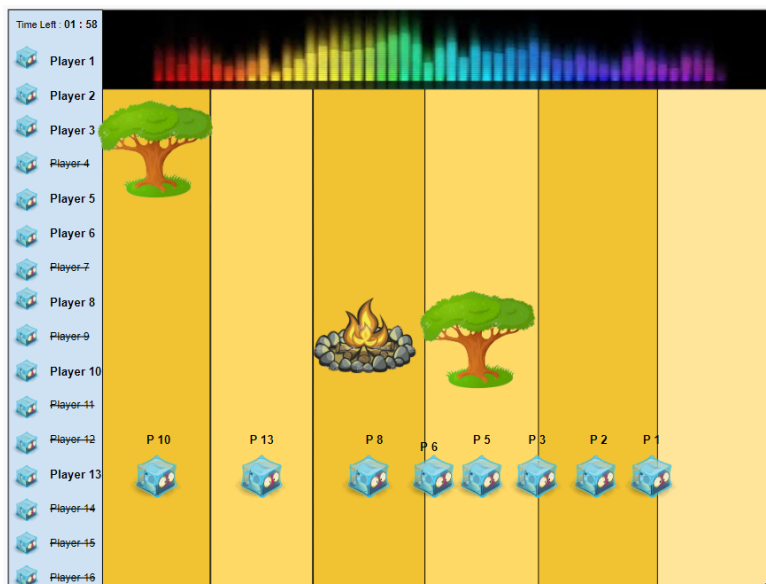


I. Présentation du jeu :

1. Introduction

“Run' N Swipe” est un jeu de type “runner” en 3D, où le but est de finir la course en évitant tous les obstacles. Il se joue aussi bien en solo, qu’en multijoueur jusqu’à 16 joueurs simultanés et de manière locale, grâce à la création d’un salon depuis votre ordinateur. Les joueurs sont libres d’évoluer dans leur couloir associé ou bien de se déplacer de couloir en couloir afin d’aller gêner leurs adversaires et ainsi les éliminer. Avant chaque course, les joueurs choisissent une musique qui déterminera la carte mais également la multitude d’obstacles qui seront générés de manière procédurale par l’ordinateur, afin que la présence de ces derniers soit en accord avec la musique sélectionnée précédemment. A la fin de chaque course, le gagnant ou les gagnants remporteront un point dans le classement fait entre tous les joueurs présents.

Une fois une manche terminée, un joueur peut quitter ou rejoindre le salon déjà existant en utilisant son propre smartphone, ceci en scannant le QR Code propre au salon. De cette manière, chaque joueur contrôle son personnage à l’aide de son smartphone, tel une manette. Maintenant, plus aucune raison pour ne pas inviter ses amis.



Ci-dessus, un schéma approximatif de la vue représentant une course constituée de 16 joueurs. Il s'agit de l'affichage présent sur votre écran d'ordinateur ou votre téléviseur.



2. Fonctionnalités

La caméra :

La caméra est une caméra fixe permettant une vue globale sur le niveau, en plongée.

Les biomes :

Les joueurs pourront choisir un biome parmi la forêt, le temple, le donjon, le volcan, etc. Liste non-exhaustive ([cf. GDD](#)).

Les capacités et contrôles :

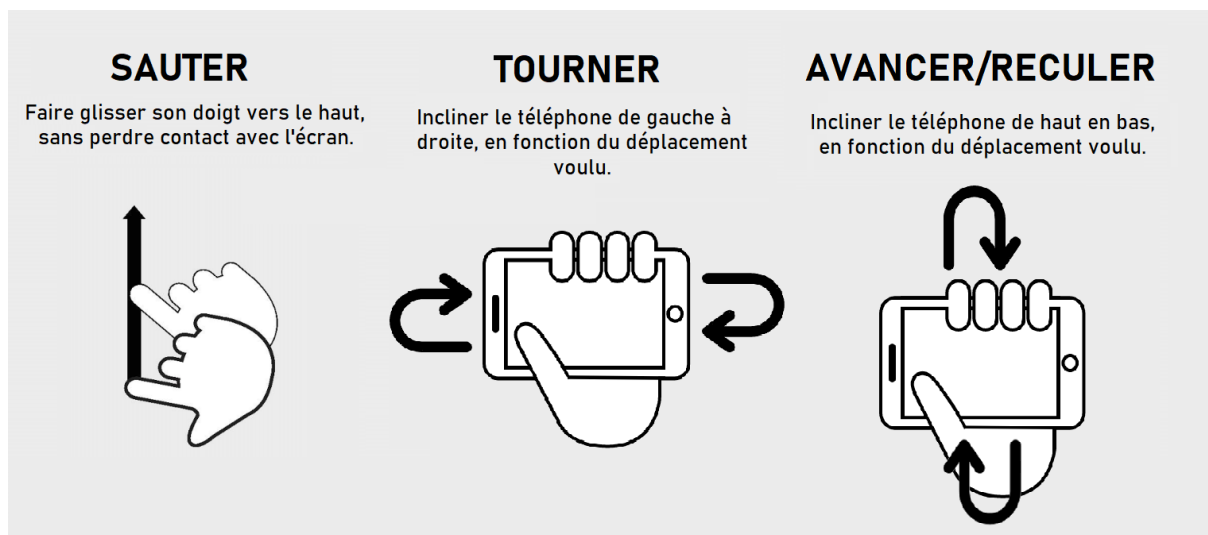


Figure 2 : Capacités de contrôle sur smartphone

Les obstacles sont scindés en deux types : les obstacles qui sont franchissables et ceux qui ne le sont pas.

Les obstacles franchissables peuvent être traversés en effectuant un mouvement, tel que sauter par-dessus ou se baisser pour passer en dessous.

Les obstacles infranchissables ne peuvent qu'être évités, sous peine de perdre une vie.

Cependant, lorsqu'un joueur se bute à un obstacle, franchissable ou non, ce même joueur perd une vie et devient invulnérable pendant 3 secondes. De plus lors de la



prise d'un obstacle le personnage se retrouve projeté en arrière sur une distance qui dépend de la vitesse à laquelle il s'est pris l'obstacle.

3. Univers

L'univers du jeu est fantastique et cartoonesque. On inverse les règles, le joueur n'incarne plus ce fameux héros qui se doit de tuer des monstres plus faibles que lui pour pouvoir évoluer. On incarne cette innocente créature qui se fait pourchasser par ce "méchant" héros qui vous tuera s'il vous attrape. Ce qui nous amènera à fuir dans de multiples et divers lieux emblématiques des jeux dits "Role Playing Game", tels que les plaines remplies de verdure, mais également à travers un château refermant moult trésors.

II. Plateformes et spécifications hardware

Configuration moyenne utilisée par la plupart des utilisateurs :

- Un ordinateur possédant un système d'exploitation Windows 8 à 10.
- Smartphone(s) : pour le moment, seul le système d'exploitation Android est disponible.

Nous préférons dans un premier temps, pour les ordinateurs, nous concentrer sur Windows pour avoir une version stable qui ensuite sera adaptée sur les autres systèmes d'exploitation. De plus, Windows est le système d'exploitation le plus utilisé.

Pour les smartphones, nous avons choisi Android, c'est le système d'exploitation le plus utilisé et il ne demande pas la possession d'un compte développeur comme iOS.

Il n'a pas encore été établi de configuration minimale et conseillée. Elle sera testée, plus en détails, une fois le jeu davantage avancé. En revanche, pour ce qui est de la compatibilité de l'application, des tests ont été effectués sur différents appareils lors de nos playtests. En ce qui concerne les ordinateurs : notre jeu fonctionne parfaitement sur Windows 8 à 10, peu importe la machine utilisée.

Concernant les téléphones, voici la liste des appareils testés, ainsi qu'une couleur représentant la compatibilité, "rouge" pour un test non-concluant (souci de compatibilité, de mise en route ou tout autre problème empêchant le bon fonctionnement du jeu) et "vert" signifiant une utilisation normale de l'application.



Téléphone	Compatibilité
MI9 Xiami	
Huawei Y5	
Redmi note 7	
Huawei P9 Light	
Redmi 7	
Huawei P20	
Xiami 9	
Redmi 8 Xiami	
Samsung J3	
Xiami Redmi 8T	
Asus X00LD	
Samsung Galaxy S9	
Samsung Galaxy S10	
Samsung Note	
Samsung S20	
Huawei P8 Lite 2017	
Honor 10	

Figure 3 : Tableau récapitulatif des tests de compatibilité des téléphones

III. Outils techniques utilisés

1. Environnement de développement

Voici la liste des outils utilisés lors de ce projet :

- Visual Studio 2019 : Environnement de développement.
- JetBrains Rider 2021.1.2: Environnement de développement
- GIT: outils de versionning
- Gitlab: hébergement du projet en ligne
- Blender 2.82 et Maya (Autodesk 2020) pour la création d'objets 3D.
- FruityLoops Studio, Audacity 3.0 et Reaper 6.29 pour le sound design.
- Adobe Photoshop 2021 pour les créations artistiques (logos, images, textes).

2. Technologies

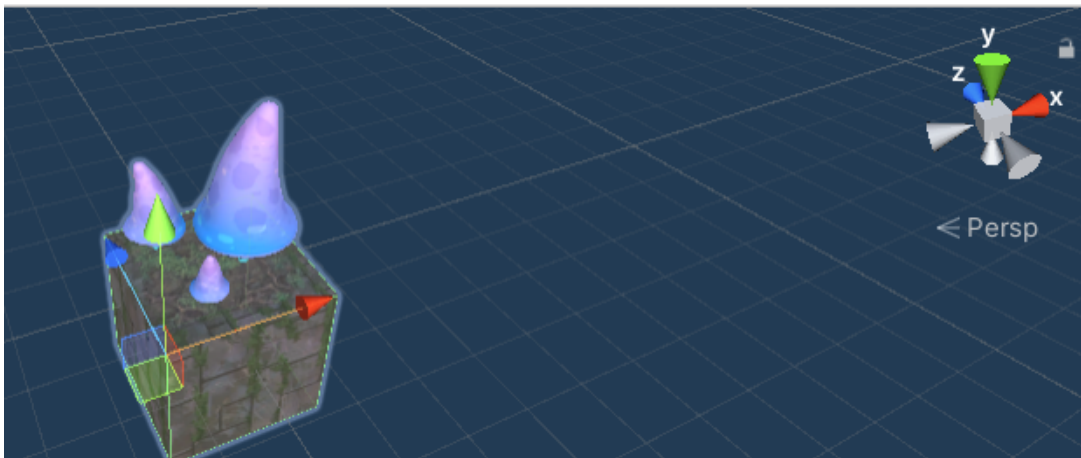
- Le moteur de jeu utilisé est Unity, version 2019.3.5f1
- Le langage de programmation utilisé est le C#, utilisé dans Unity.

IV. Pipeline de production

1. Production d'un modèle 3D

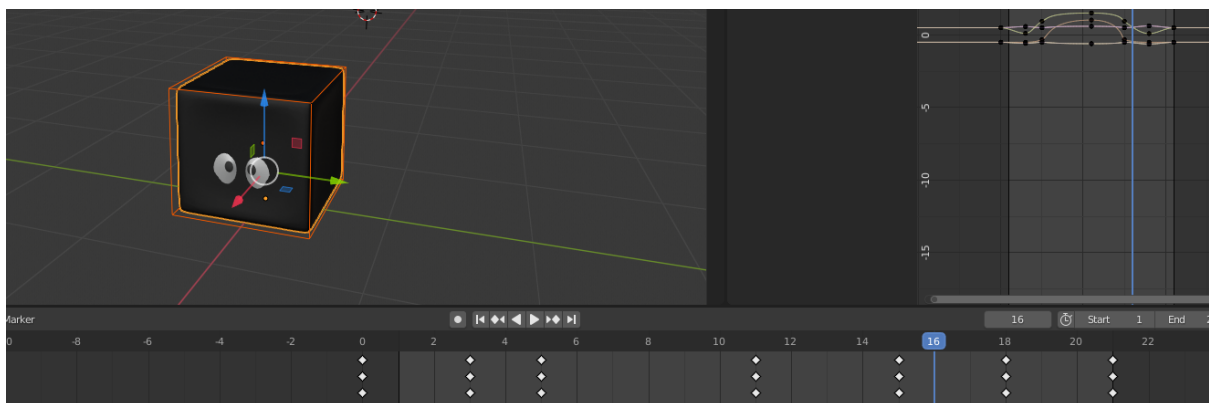


- Réflexions à propos du modèle (Recherche d'inspiration, etc.)
- Modélisation du modèle sous Maya/Blender
- Texturisation & Gestion des UV du modèle sous Maya/Blender
- Disposant des points de pivots, le point de pivot du nœud du parent doit être à une valeur minimale en z et en x, au niveau du sol.
- Export en .FBX du modèle
- Intégration dans Unity du modèle, ainsi qu'un ajout d'animation si nécessaire directement depuis Unity, de même pour les effets visuels, tels que les particules.



2. Production des animations

- On import le modèle du slime dans Blender
- Utilisation de l'outil d'animation de Blender
- On modifie le slime selon la forme que l'on veut lui donner à un moment clé
- On ajoute une Keyframe à chaque moment clé
- On export l'animation sur Unity
- L'utilisation de bones n'est ainsi pas nécessaire pour le saut



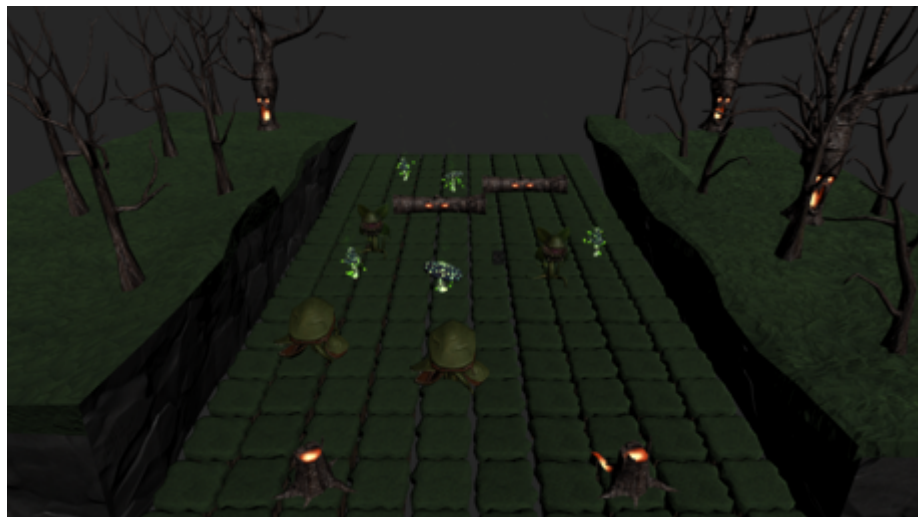
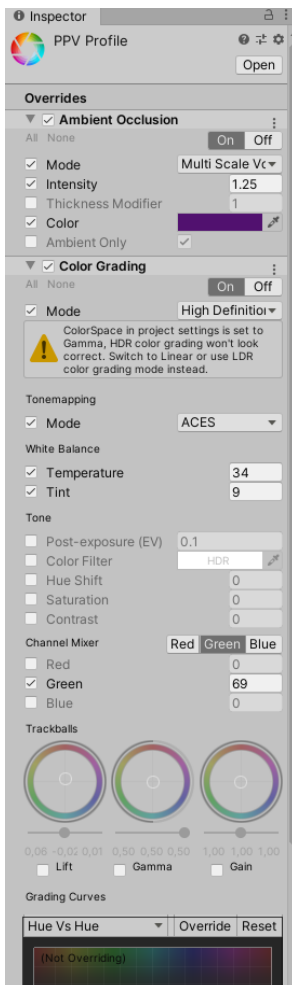


3. Production d'un effet sonore

- Récupération des sons sur internet (Freesound.org le plus souvent)
- Retouche du son sur Reaper (Tronquage, Mixage, réduction des sons, pitch)
- Implémentation directement dans Unity

4. Production du post-processing

- Utilisation du Post Processing 2.3
- Besoin d'un volume où l'on renseigne le PPV Profile
- On ajoute différents effets que l'on applique ensuite à la caméra
- On ajoute un effet plus sombre, et les couleurs vives ressortent plus





V. Outils spécifiques

- Text Mesh Pro : Plug-In permettant de gérer le texte de l'UI avec un plus large éventail d'outils que celle de Unity comme la police, les couleurs, les effets.

VI. Librairies Externes

1. Partie Réseau

Suite à la dépréciation de Unet, il a fallu s'orienter vers une autre solution pour le jeu en réseau. Au départ, nous étions partis sur photon (PUN), permettant à l'aide d'un cloud de connecter jusqu'à 20 joueurs gratuitement. Cette limitation nous a dérangés et nous avons préféré chercher une autre solution, car cette dernière aurait pu empêcher l'utilisation de l'application de manière simultanée et maximale.

MLAPI : MLAPI est une bibliothèque MidLevel, c'est-à-dire qu'elle permet la programmation à haut niveau, avec possibilité de descendre à bas niveau ce qui ne nous bridera pas lors de nos développements. De plus, MLAPI propose un système d'approbation de connexion afin d'autoriser ou non un client à se connecter à la partie selon des données d'entrée. Contrairement à Miror, il a, paraît-il, une bien meilleure gestion de la mémoire. Le risque étant bien évidemment davantage côté serveur et du réseau qui doit encaisser jusqu'à 16 connexions simultanées, avec chaque smartphone qui sert de relais pour transmettre les inputs du joueur.

Voici un comparatif entre MLAPI, Miror et HLAPI au niveau des features : <https://mlapi.network/features/>

Pun : Comme dit précédemment, la limitation à 20 joueurs nous a posé problème.

Mirror : Mirror est une amélioration de Unet, en revanche, d'après nos recherches, il apparaît comme ayant une très mauvaise gestion de la mémoire, imposant une hiérarchie de codage, et utilisant Telepathy comme système de transport. L'inconvénient est que Telepathy utilise 2 threads par client (une écriture, une lecture), ce qui n'est pas recommandé (recommandation de Microsoft).

Après analyse de l'existant, PUN, Miror et MLAPI pour les plus connus, nous avons décidé de choisir MLAPI.



2. Partie Transport

Toujours suite à la dépréciation de Unet, il a fallu chercher une autre couche de transport que la couche Unet. Notre choix s'est donc porté sur LiteNetLib parmi celles-ci : Ruffles, Enet, SteamP2P, LiteNetLib et WebSockets, qui sont les couches de transport compatibles avec MLAPI.

Ruffles : Ruffles est une bibliothèque UDP fiable et légère très récente.

Enet : Enet est une bibliothèque en développement depuis 2002 et simple d'utilisation. Elle fournit des fonctionnalités classiques par-dessus UDP, c'est une solution connue et robuste notamment utilisée dans League of Legends. Malheureusement, elle ne fonctionne que sur Windows, Linux et MacOS.

SteamP2P : SteamP2P ne fonctionne que sur Windows, Linux et MacOS en plus d'être expérimentale, nous ne l'avons donc pas choisi.

LiteNetLib : LiteNetLib est une bibliothèque UDP légère et fiable native de C# compatible avec Windows, Linux, MacOS, Android, IOS, UWP, et Lumin OS. Elle ne nécessite pas d'autre installation annexe.

WebSockets : WebSockets fonctionne sur le protocole TCP/IP utilisé pour les navigateurs modernes. Il est compatible avec toutes les plateformes mais il est expérimental.

Notre choix s'est donc fait entre LiteNetLib et Ruffles (car ils sont les seuls compatibles Windows & Mobiles). Puis notre choix s'est porté sur LiteNetLib.

3. Partie Génération Maps

DSPLib : DSPLib est une bibliothèque qui apporte des solutions complètes pour faire de la transformation de Fourier rapide (FFT) avec .NET 4.

4. Partie UI

TextMesh Pro : Outil de gestion des textes, afin de les améliorer et remplacer ceux par défaut d'Unity.



VI. Architecture Logicielle

Le jeu fonctionne via une architecture événementielle, couplée par une architecture MVC pour la gestion autonome des pages UI. Mis à part l' "Event Manager" et le "Messaging Manager", utilisés pour la propagation d'événements, toutes classes "non manager", ne possèdent aucun accès aux managers. Le seul moyen de communiquer étant les événements. Les managers peuvent communiquer entre eux, cependant cela reste à éviter.

Voici les différents managers expliqués de manière individuelle, en séparant les managers de l'application téléphone aux managers de l'application PC (Serveur).

1. Coté Serveur

- **Server Camera Manager :**
Permet à l'instar du Server Menu Manager, d'utiliser la bonne caméra selon la situation. En effet, il existe 3 caméras, une caméra pour les menus, une caméra pour le jeu et enfin, une caméra lorsque le jeu est en pause.
- **Messaging Manager :**
Permet au serveur d'envoyer des messages aux clients connectés. Les messages transmis sont des événements que nous propageons sur le téléphone.
- **Server Network Manager :**
Manager gérant le serveur. C'est-à-dire son initialisation, sa fermeture et l'approbation de nouveaux joueurs ainsi que leurs déconnexions.
- **Network Manager :**
Manager obligatoire de MLAPI, la bibliothèque réseau utilisée, contrôlant si 2 versions du jeu sont compatibles, le port à utiliser et gérant également l'envoi de messages via le protocole UDP LiteNetLib.
- **Sfx Manager :**
Permet de jouer les bruitages du jeu.



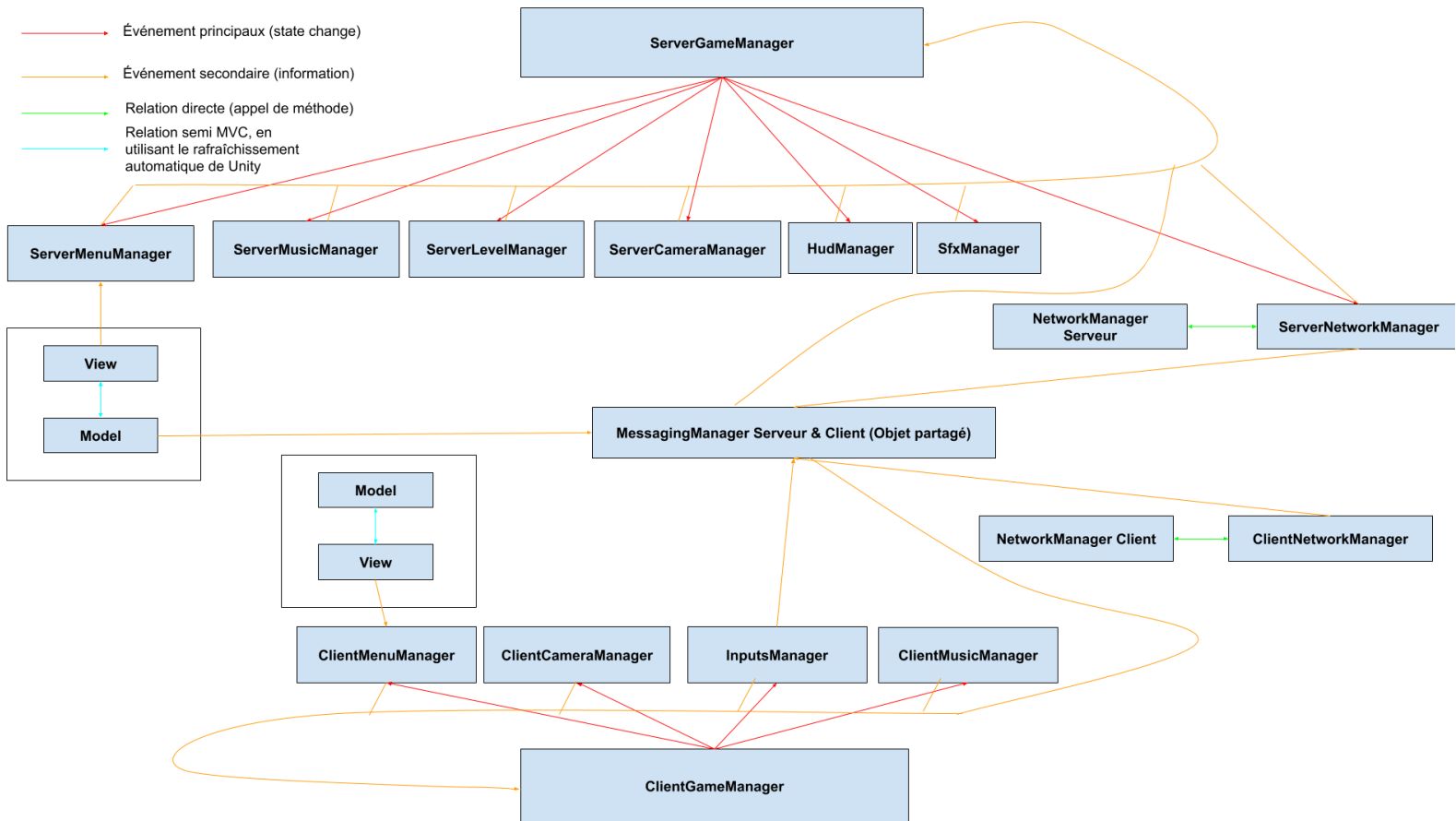
- **Server Music Manager :**
Permet le contrôle de la musique de fond jouée.
- **Level Manager :**
Permet de contrôler la génération procédurale de la carte via la musique
cf : [génération procédurale](#)
- **Server Menu Manager :**
Permet d'utiliser le bon menu selon la situation et de notifier via un ou plusieurs événements les différents boutons, où le joueur a interagi.
- **Hud Manager :**
Permet de contrôler et de mettre à jour, le HUD en jeu sur le serveur.
- **Server Game Manager :**
Manager principal de tous les managers, unique émetteur des événements principaux permettant de changer le jeu d'état, tel que le fait de changer de menu, ou encore de passer en jeu ou en pause, etc.

2. Coté Client

- **Network Manager :**
Manager obligatoire de MLAPI, la bibliothèque réseau utilisée, contrôlant si 2 versions du jeu sont compatibles, le port à utiliser et gérant également l'envoi de messages via le protocole UDP LiteNetLib.
- **Client Network Manager :**
Manager gérant la connexion et la déconnexion à un serveur, ainsi que le stockage de l'id du joueur attribué par le serveur.
- **Inputs Manager :**
Manager gérant, lorsqu'il est demandé, l'écoute des différentes actions du joueur sur son téléphone avant de les transmettre au serveur.
- **Client Menu Manager :**
Manager contrôlant l'affichage des menus ainsi que la retransmission sous forme d'événements au reste des managers lorsqu'un bouton est cliqué sur le menu courant.
- **Client Game Manager :**
Manager principal de tous les managers, unique émetteur des événements principaux permettant de changer le jeu d'état, tel que le fait de changer de menu
- **Client Camera Manager :**
Manager contrôlant la caméra qui doit être utilisée.
- **Client Music Manager :**
Manager contrôlant la musique d'ambiance utilisée.

VII. Techniques de rendu

16





VIII. Budgets

- Notre jeu ne devra pas excéder 100 mo sur le téléphone, car les modèles 3D ne lui sont pas nécessaires.
- Sur l'ordinateur, le jeu ne sera pas supérieur à 5 giga octets.
- Les modèles 3D devront rester en "low poly", pour un nombre cible de polygones, maximum dans la scène, d'environ 500 000.
- Le nombre d'objets dans la scène ne devra pas outrepasser, à peu près 500.

Toutes ces conditions sont nécessaires pour que l'application fonctionne sur un maximum de machines possibles. Les cibles sont autant les ordinateurs peu performants que ceux extrêmement optimisés, de même pour les téléphones, afin que personne ne soit lésé. La limite de 500 000 polygones permettra d'alléger la carte graphique tandis que la limite de 500 objets maximums allégera le processeur.

IX. Performances

Nous visons, au minimum, 60 images par seconde afin que le jeu soit le plus fluide et agréable possible. Cela permettra une visibilité optimale du jeu.

X. Mécanismes de gameplay

1. Génération de la map LD (Level Design)

En ce qui concerne la génération de la map LD, lors de son développement, nous avons imaginé deux possibilités :

- Créer les maps manuellement, à travers un éditeur de map
- Générer les maps de manière procédurale grâce à un algorithme

La première option demande la création d'un éditeur de maps, comme par exemple dans le jeu OSU!.



Cependant notre volonté n'est pas d'obliger les joueurs à passer du temps sur le jeu autre que dans une partie entre amis. Nous avons donc conclu que l'édition de map n'était pas la bonne solution, car cela était trop chronophage mais peut également être redondant et fastidieux. Nous avons donc opté pour la deuxième solution.

La génération procédurale des maps demande d'analyser les musiques, pour en tirer des informations nécessaires à la création des maps. Pour convertir la musique en données exploitables. Nous avons donc été contraints d'utiliser l'algorithme de [transformation de Fourier rapide](#) (Fast Fourier Transform ou FTT, en anglais) afin de transformer des données discrètes du domaine temporel dans le domaine fréquentiel et donc d'obtenir le spectre de la musique, exprimé en hertz.

Dans un premier temps, nous avons cherché à trouver le battement par minute (bpm) de la musique grâce à son spectre et/ou à l'aide d'API de musique permettant de récupérer des informations sur les musiques, dont son bpm notamment. Après plusieurs essais, nous avons abandonné cette solution car cela rendait la fréquence d'apparition des obstacles beaucoup trop monotone, et donc lassant à force.

Dans un second temps, nous avons décidé de nous pencher sur les "moments forts" de la musique. Ces derniers sont obtenus à l'aide du spectre de la musique, sur lequel nous nous contentons de seulement garder les valeurs audibles par l'humain (entre 20 et 20 000 hertz). Grâce à ces valeurs obtenues, nous pouvons donc définir une moyenne des pics obtenus, permettant de définir si un moment de la musique est un moment fort ou non. Nous stockons ces moments forts (en hertz) dans une liste qui contiendra les potentiels obstacles. Elle nous servira, une fois rentrée dans notre **algorithme**, à déterminer le pattern de la musique.



2. L'Algorithme

C'est donc l'algorithme qui va définir la map LD grâce à la liste contenant tous les pics récupérés à l'aide de la formule mathématique FFT, vu précédemment.

L'algorithme est initialisé ainsi, nous cherchons tout d'abord le plus haut pic atteint et nous additionnons la totalité des pics afin d'en obtenir une moyenne. Une fois, le maximum et la moyenne obtenus, on cherche la valeur du pic, dans la liste triée par ordre croissant, correspondant au pic situé à 20% dans cette liste. Puis avec cela, on définit le seuil maximum d'apparition d'obstacles selon la formule temporaire suivante :

Le seuil d'écoute d'un pic est : Moyenne / (Maximum + la valeur située à 20%)

Un moment fort est ce que la bibliothèque d'analyse de flux audio qu'on utilise spécifie comme étant un pique dans le graphique de la musique et donc tout "moment fort" possédant un pic supérieur à cette valeur, sera écouté. Cela permet plusieurs choses :

- Réduire le nombre d'obstacles écoutés, sinon celui-ci est trop élevé
- Plus la musique possède un "maximum" violent, plus le chiffre sera bas et rendra l'algorithme sensible, il trouvera donc plus de moment fort. Chaque moment fort permet la génération d'un obstacle, on aura donc plus d'obstacles.

Suite à cela, nous initialisons la vitesse de déroulement de la carte. Pour ce faire, nous faisons la division du nombre de "moment fort" par le temps de la musique (la formule est toujours en évolution). La vitesse de déroulement de la carte sera donc fixe pendant la musique. Vous pouvez voir dans l'annexe l'ancienne méthode utilisée pour le déroulement de la carte, elle modifiait la vitesse pendant la musique en fonction des obstacles sur la map. Nous avons décidé de changer car la modification de la vitesse de déroulement était dérangeante pour les joueurs.

La map LD, où se trouve les joueurs, est constituée de blocs, 10 de largeur et 25 de longueur. Dans un premier temps, nous avons généré des obstacles lors de l'apparition de "moments forts" dans la musique. Après de multiples tests, nous avons remarqué que cette génération procédurale rendait la map LD non lisible. Nous avons donc choisi une autre méthode pour la génération des obstacles. Pour plus d'informations sur la première méthode, vous pouvez consulter les annexes. Dans la nouvelle méthode, nous avons décidé d'utiliser des patterns comme obstacle pour que le joueur puisse facilement identifier un chemin. Un pattern possède comme caractéristique une longueur, une largeur et une difficulté. Lors d'un



“moment fort”, l'algorithme va vérifier s'il peut introduire un nouveau pattern, si un pattern est toujours en cours, l'algorithme va ignorer le “moment fort” et ne pas poser de pattern.

Pour placer un pattern, l'algorithme va vérifier la difficulté de la musique suivant la formule suivante: nombre de moment fort présent dans la musique divisé par le temps en seconde de la musique. Suite à cela, l'algorithme accédera aux patterns présents dans la map LD qui ont une difficulté proche de celle de la musique. La difficulté des patterns est décidée arbitrairement par la personne qui a créé le pattern en question.

Nous souhaitons à la fin du projet que l'algorithme se lance lors de l'ajout d'une musique dans le jeu. Cela permettra de définir une difficulté à la musique mais aussi de stocker les maps LD. Run' N Swipe étant un jeu multijoueur local, nous pensons qu'il est nécessaire de le rendre fluide pour ne pas perdre l'ambiance qu'il génère lors des courses. Le fait de stocker les maps LD permet au jeu de lancer directement la course après la sélection de la musique et de ne pas avoir de temps de chargement qui peut casser l'ambiance et la dynamique de jeu.

XI. HUD

Il n'y a pour l'heure, aucun HUD dans le jeu, ni côté serveur, ni côté client. Voici un schéma explicatif de nos idées sur celui présent côté serveur :

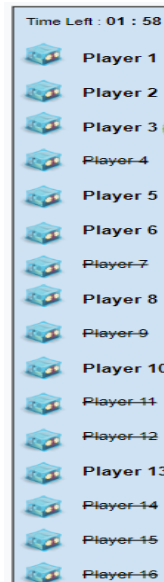


Figure 7 : Exemple de HUD

Celui-ci sera composé de cette bande sur le côté, gauche ou droit, de l'écran afin de déranger le moins possible les joueurs, celui-ci comportera le temps restant et le nombre de joueurs encore dans la course.

Pour ce qui est du HUD côté client, nous avons potentiellement pensé à un aperçu du slime du joueur. Cependant nous désirons y afficher très peu d'informations sur le jeu afin de ne pas obliger le joueur à devoir regarder son téléphone et ainsi perdre le fil sur l'écran principal.



XII. Menus

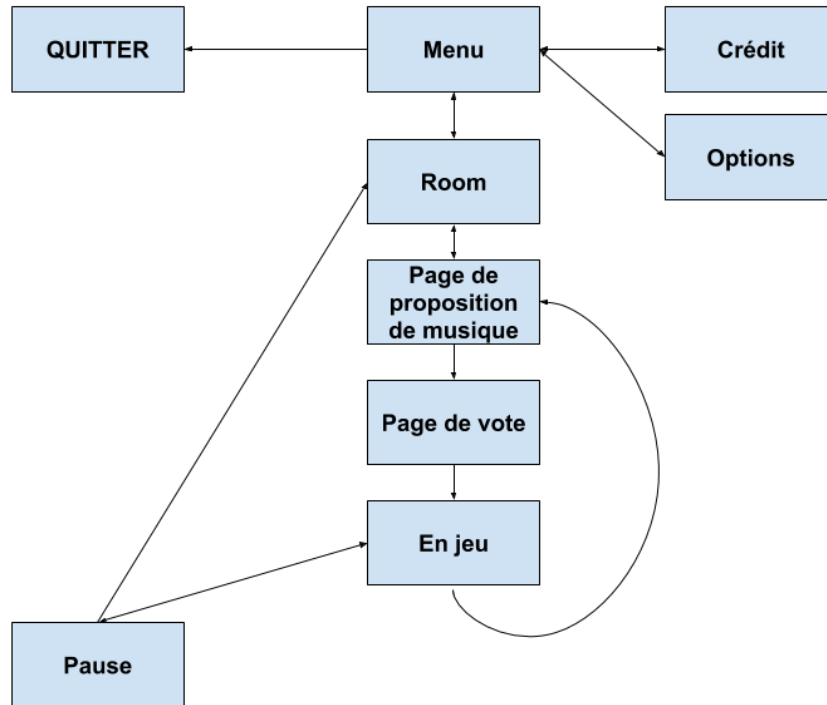


Figure 8 : Liaison entre les différentes parties du menu principal

Première version des menus, pour l'intégralité des différentes vues cf. [GDD](#) :





Figure 10 : Menu de sélection du personnage sur portable

XIII. Audio

Excepté la musique du menu qui est une chanson libre de droits, étant donné que nous utilisons directement les musiques sur les téléphones des joueurs, il n'y a pas de travail sur la musique d'ambiance en elle-même. Seuls les effets sonores seront, par la suite, travaillés.

XIV. Convention d'écriture de code

- Langue du code : Anglais // Commentaires en Français
- Convention écriture des attributs : PascalCase
- Convention des variables locales : camelCase
- Convention d'écriture des champs static readonly ("final") : MAJUSCULE, séparée par des underscores
- Convention d'écriture des fonctions et des méthodes : MyFunction()

XV. Équipe et organisation

1. Équipe

- Lucas DERAEDT : Développeur et UX designer
- Sullyvan MODESTE : Sound designer et modélisateur 3D

2. Organisation

Versionning du code grâce à Git, et utilisation de Gitlab pour le partage de ce dernier. Mise en place d'un google drive, pour les échanges de fichier annexe, tel que le logo ou encore les maquettes pour les menus.

XVI. Risques techniques

Les risques techniques qui ont été identifiés jusqu'à maintenant sont :

Risque : Une génération procédurale et déterministe de chaque carte en fonction de la musique sélectionnée.

Solution : Concernant les détails des paramètres pris en compte pour la génération de la carte, tous les détails se trouvent [ici](#).

Risque : Le bon fonctionnement du jeu en réseau local avec des connexions simultanées de 16 joueurs.

Solution : L'utilisation de la librairie réseau MLAPI ainsi que du protocole LiteNetLib, expliqués de manière détaillée [ci-dessus](#). De plus, nous devons trouver le juste équilibre au sujet du nombre de messages envoyés au composant servant de serveur.

Risque : La possibilité à chaque smartphone, sous n'importe quel type de version d'android, de rejoindre la partie et ainsi être utilisée comme manette par le joueur.

Solution : L'utilisation d'un QR code généré par la partie "SERVEUR" gérée par l'ordinateur ainsi que d'une application créée grâce à Unity permettant le scan du QR CODE et l'utilisation du téléphone en tant que manette (partie "CLIENT"), cf [GDD](#).

Risque : Impossibilité de scanner le QR Code pour rejoindre la partie



Solution : Dans le cas échéant où le smartphone n'arriverait pas à scanner le QR Code, une option manuelle est disponible où l'on rentre l'adresse IP à la main.



Figure 11 : Salon d'accueil d'une partie

Risque : La lisibilité du jeu lorsqu'il y a 8 joueurs présents sur la carte, afin que chacun puisse se repérer sur cette dernière.

Solution : La personnalisation du personnage est propre à chaque joueur. Personnalisation qui s'effectue par le biais d'un couvre-chef et d'une couleur. Chaque Slime doit être unique et donc il est rendu impossible pour les joueurs de posséder un slime identique en tout point.



Figure 12 : Salon de personnalisation du personnage

Risque : Risque matériel dû à l'architecture réseau, si la livebox utilisée est défectueuse ou si la connexion à celle-ci est perturbée, cela risque de causer de la latence entre les actions des joueurs.

Solution : Jouer depuis un réseau de proximité, tel qu'un partage de connexion avec son téléphone.

Risque : Impossibilité de se reconnecter à la partie, si le joueur ferme ou revient au menu du téléphone par erreur de manipulation.

Solution : Créer un système de reconnexion.



Risque : L'écran du téléphone se met en veille si le joueur ne fait aucune action autre que des déplacements. Le joueur ne deviendrait alors plus contrôlable le temps de déverrouiller à nouveau le téléphone.

Solution : Désactiver le mode veille du téléphone pendant l'exécution du jeu.

Risque : Dans nos évolutions possibles, le serveur pourrait très bien devenir une tablette ou encore un autre téléphone portable. Pour cela, nous avons besoin que notre jeu tourne sur des machines peu puissantes.

Solution : Limiter le nombre d'objets dans la scène ainsi que le nombre de polygones.

XVII. Fonctionnement GIT

On utilise gitlab pour l'hébergement, on peut donc avoir un projet privé et avoir un stockage plus important que gitHub.

Pour ce qui est de la gestion des versions, on possède une branche en commun sur laquelle on envoie nos modifications chaque semaine pour la tenir à jour.

Dans les cas de travail un peu plus complexe et qui pourrait potentiellement créer des crashes critiques, on crée une nouvelle branche et une fois terminé soit on fusionne les deux branches soit on récupère les changements de cette branche sur la principale. Dans les deux cas, on a les traces présentes sur gitlab.



Annexes

Ancienne vitesse de défilement :

Suite à cela, nous actualisons la vitesse de déroulement de la carte. Pour ce faire, nous faisons la moyenne des intensités sonores sur un court laps de temps en regardant également dans le futur afin que la vitesse soit adaptée à ce qui approche et non pas à ce qui est passé.

La formule est la suivante :

- t = temps écoulé depuis la dernière mise à jour
- Nouvelle vitesse = $t * \text{la vitesse minimale des obstacles} * (\text{la vitesse actuelle} + \text{la moyenne des intensités depuis la dernière mise à jour} + \text{la moyenne des intensités dans le futur sur un temps } t) / 3$

Cette formule fut trouvée suite à de nombreux tests et est amenée à évoluer. Voici un extrait du tableau excel construit en récoltant des données sur différentes musiques, qui nous a permis d'établir cette formule en cherchant des corrélations entre les données.

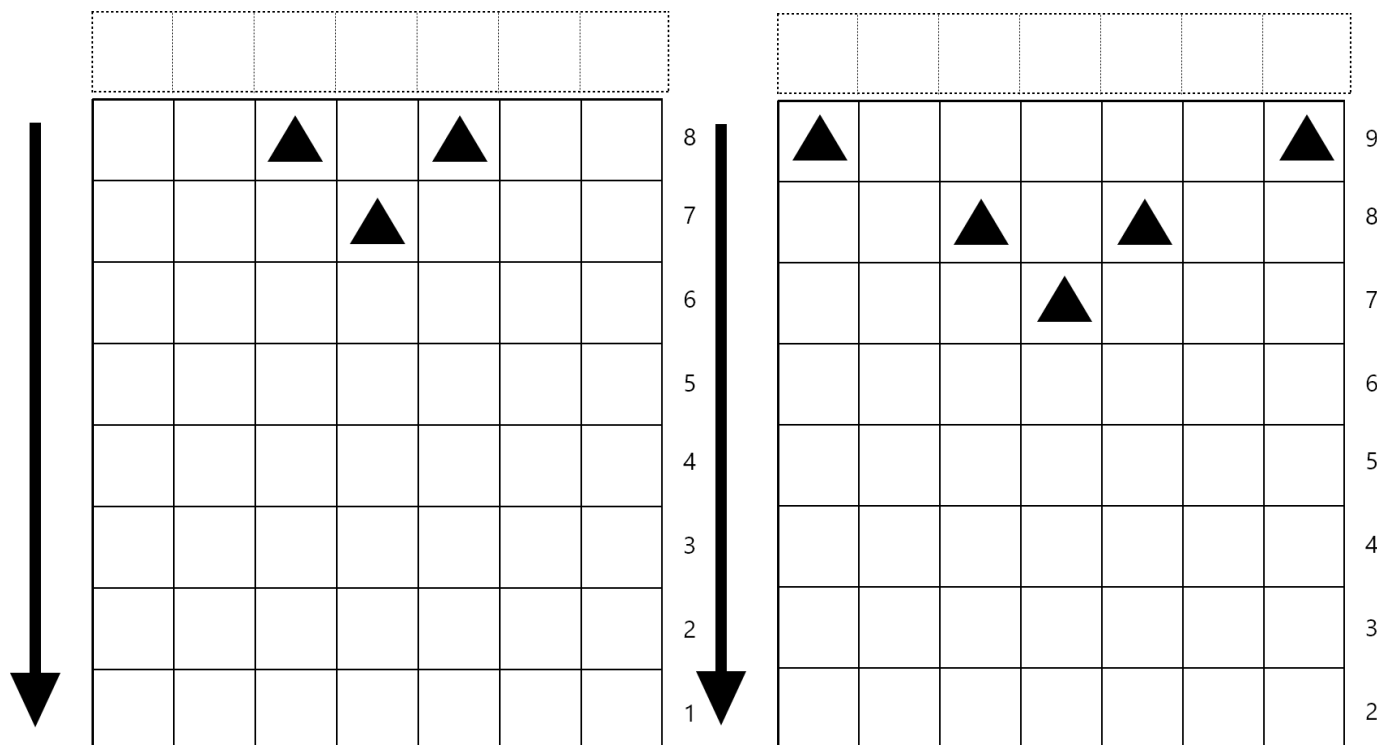
Song	Nombre de beats	Moyenne pruned	Medianne pruned	Max pruned	Difficulté expectée	Fréquence	Durée seconde	Tempon Moyen	ThresholdMoy	ThresholdMax	ThresholdMed
Blonde	801	0,95	0,51	4,44	Facile-Moyen	44100	188	255,63829787234	0,78	1,4	0,8
Axel Theself	2168	0,76	0,49	5,825	Facile-Moyen	48000	431	301,809744779582	0,53	1,13	0,6
Papaya	905	0,8	0,51	4,7	Difficile	48000	235	231,063829787234	1,7	2,44	1,73
Fais de ton mieux Gerald de palmas	1036	0,6	0,427	3,1	Facile	48000	231	269,090909090909	0,63	1,23	0,67
Le chant des druides – Manau	1354	0,77	0,42	5	Moyen	48000	256	317,34375	0,69	1,21	0,74
Infinite power – FatRat	944	0,94	0,58	4,39	Difficile	48000	208	272,307692307692	0,95	1,45	1,03
Le chien du forgeron – Manau	1183	1,17	0,7	6,12	Facile	48000	263	269,885931558935	0,53	0,91	0,56
Marnik Up & Down	775	0,5	0,29	3,29	Facile	44100	163	285,276073619632	0,65	1,3	0,65

Nous en avons déduit que l'intensité des sons (à droite) était la meilleure variable pour la vitesse puisqu'elle corrélait dans la majorité des cas, à la vitesse attendue, contrairement au pic fréquence.



Map et obstacle :

La map LD se décompose de la manière suivante :



Elle est composée de X lignes, numérotée de 1 jusqu'à 8 ci-dessus (schéma de gauche). Chacune de ces lignes est composée de X carrés. Chaque carré représente un potentiel emplacement d'obstacle. Les obstacles n'apparaissent que sur la dernière ligne, ici en pointillés. La map LD fonctionne tel un tapis roulant, ainsi les personnages n'avancent pas réellement, c'est la map LD qui avance : l'algorithme génère la dernière ligne en la concaténant à l'ancienne dernière ligne. Le terrain avance en continu, et une fois la plus vieille ligne allée trop loin, une nouvelle ligne se génère. Cela n'a pas toujours été ainsi.

En effet, à cause de la vitesse des obstacles qui dépend de "Time.deltaTime", la vitesse des obstacles n'est pas constante. Ainsi, si nous générons un obstacle en utilisant une coroutine qui demanderait une génération tous les x temps, temps calculé selon la vitesse et la distance à parcourir, nous obtenons des décalages dans le sol. Nous avons testé bien des méthodes et avons conclu que la meilleure était celle que nous utilisons aujourd'hui, c'est-à-dire mémoriser l'objet le plus récent et le plus proche de la fin du parcours à chaque génération, pour l'utiliser comme repère dans l'apparition de la ligne suivante.



Enfin, les obstacles ne se positionnent pas tout à fait aléatoirement, on établit une liste regroupant tous les patterns, ce sont des briques de level design contenant des obstacles, ils font toute la largeur de la carte mais ont des hauteurs qui peuvent varier. Ils sont faits à la main et servent à limiter la difficulté du level design de cette manière on est sûr qu'il y a toujours un chemin facilement accessible.

Pour savoir quelle longueur et obstacles choisir, l'algorithme se base sur la musique dans un premier temps, on détermine le rythme de la musique plus les temps forts sont proche plus l'algorithme pioche dans les patterns les plus petits et inversement plus les temps forts s'éloignent et la musique se calme plus les patterns choisis seront grand. En enchainant les plus petit patterns on augmente le nombre d'obstacle présent sur la carte et plus on a de grands patterns moins il y a d'obstacle et donc plus il est facile de les esquiver.